

quest_teleop: Control Logic and Math

UR5 + Robotiq Quest 3S teleoperation package

June 17, 2026

Abstract

This document specifies the control math implemented in the `quest_teleop` package (`src/quest_teleop/`). The mapping from Meta Quest 3S controller poses to UR5 motion follows the DROID VRPolicy formulation. One backend-agnostic control loop (`teleop.py`) drives two interchangeable robot backends: a direct `ur_rtde` backend (real robot / URSim) and a ROS 2 backend (`/forward_velocity_controller`, for the `launch_all.sh` fake-hardware simulation). The two backends share *identical* control math; they differ only in how the final Cartesian command is realised (Section 6).

Contents

1	Notation and frames	1
2	Forward reset: yaw-only alignment	1
3	VR pose low-pass filter	2
4	DROID target pose	2
5	Control modes	2
5.1	Speed mode (<code>--mode speed</code> , default)	2
5.2	Servo mode (<code>--mode servo</code> , RTDE only)	3
6	Backend realisation	3
7	Safety	3
8	Symbol / flag / code reference	4

1 Notation and frames

Quaternions are scalar-first, $\mathbf{q} = (w, x, y, z)$, unit norm, acting as rotations via the Hamilton product $\otimes$. The conjugate is $\mathbf{q}^* = (w, -x, -y, -z)$. The map `rotvec( $\mathbf{q}$ )` $\in \mathbb{R}^3$ returns the axis-angle (rotation) vector and `rotvec-1` its inverse (`quat_to_rotvec`, `rotvec_to_quat`).

VR raw frame

$H^{\text{raw}} \in SE(3)$: the controller pose as reported by `oculus_reader` (a 4×4 homogeneous matrix).

Global→env remap

$M \in \mathbb{R}^{4 \times 4}$: a signed axis permutation (`vec_to_reorder_mat`) aligning the VR frame to the robot base frame. Default reorder $[-2, -1, -3, 4]$.

Alignment

$V \in SE(3)$: the “forward reset” transform (Section 2), a pure rotation.

Robot base frame

all robot poses (TCP, origins, targets) live here.

The controller pose expressed in the robot/env frame is

$$H = M V H^{\text{raw}}, \quad \mathbf{p}_{\text{vr}} = H_{1:3,4}, \quad \mathbf{q}_{\text{vr}} = \text{rmat2quat}(H_{1:3,1:3}). \quad (1)$$

2 Forward reset: yaw-only alignment

A joystick click (or the initial orientation reset) sets the alignment V so the controller’s heading maps to the robot’s forward direction. To keep gravity vertical regardless of how the controller is tilted at click time, we keep *only* the yaw (rotation about the env-vertical axis) and discard the tilt — a swing-twist decomposition.

Let the env “up” axis be $\mathbf{e}_z = (0, 0, 1)$ in the robot frame; the corresponding axis in the VR frame is $\mathbf{a} = M_{1:3,1:3}^\top \mathbf{e}_z$. Given the click-time controller rotation quaternion $\mathbf{q} = (w, \mathbf{u})$ with vector part $\mathbf{u}$, the twist about $\mathbf{a}$ is

$$\mathbf{q}_{\text{yaw}} = \frac{(w, (\mathbf{u} \cdot \hat{\mathbf{a}}) \hat{\mathbf{a}})}{\|(w, (\mathbf{u} \cdot \hat{\mathbf{a}}) \hat{\mathbf{a}})\|}, \quad \hat{\mathbf{a}} = \mathbf{a} / \|\mathbf{a}\| \quad (2)$$

(`twist_about_axis`). The alignment is the inverse yaw rotation, $V = \text{diag}(R(\mathbf{q}_{\text{yaw}}^*), 1)$, with zero translation; the translational baseline is handled separately by the origins (Section 4).

3 VR pose low-pass filter

A one-pole filter smooths the controller pose before it drives the robot (`filters.VRPoseFilter`, ported from Open-Teach). With smoothing constant $\alpha \in [0, 1)$ (`--filter-alpha`, default 0.8),

$$\mathbf{p} \leftarrow \alpha \mathbf{p} + (1 - \alpha) \mathbf{p}_{\text{vr}}, \quad (3)$$

$$\mathbf{q} \leftarrow \text{slerp}(\mathbf{q}, \mathbf{q}_{\text{vr}}, 1 - \alpha). \quad (4)$$

$\alpha = 0$ disables filtering; larger α is smoother but laggy. The first sample after a re-baseline passes through unfiltered so the origins below are captured from the same (filtered) stream.

4 DROID target pose

While the grip deadman is held, origins are latched on the first tick: the robot TCP origin ($\mathbf{p}_o^R, \mathbf{q}_o^R$) and the (filtered) VR origin ($\mathbf{p}_o^V, \mathbf{q}_o^V$). The commanded target tracks the controller’s *displacement* from its origin.

Position (with scaling).

$$\mathbf{p}_{\text{tgt}} = \mathbf{p}_o^R + s(\mathbf{p} - \mathbf{p}_o^V), \quad (5)$$

where $s = s_{\text{pos}} \cdot (s_{\text{prec}} \text{ if precision else } 1)$ (`--pos-scale`, default 1.0; precision toggled by the A/X button, `--precision-scale`, default 0.5).

Orientation. The controller’s incremental rotation is applied onto the robot origin orientation:

$$\Delta \mathbf{q} = \mathbf{q} \otimes (\mathbf{q}_o^V)^*, \quad \mathbf{q}_{\text{tgt}} = \Delta \mathbf{q} \otimes \mathbf{q}_o^R. \quad (6)$$

Equations (5)–(6) are shared by both control modes and both backends.

5 Control modes

Let the current TCP pose be $(\mathbf{p}_c, \mathbf{q}_c)$ and the control period $\Delta t = 1/f$ (`--hz`, default 50).

5.1 Speed mode (`--mode speed`, default)

A proportional controller on the pose error produces a Cartesian twist:

$$\mathbf{v} = k_p(\mathbf{p}_{\text{tgt}} - \mathbf{p}_c), \quad k_p = \text{pos_gain} \ (3.0), \quad (7)$$

$$\mathbf{e} = \mathbf{q}_{\text{tgt}} \otimes \mathbf{q}_c^* \text{ (shortest)}, \quad \boldsymbol{\omega} = k_r \text{rotvec}(\mathbf{e}), \quad k_r = \text{rot_gain} \ (2.0). \quad (8)$$

Uniform saturation. When either component exceeds its limit, both are scaled by a single factor so the screw axis (motion direction) is preserved (`limit_velocity`, after DROID):

$$\sigma = \min\left(1, \frac{v_{\text{max}}}{\|\mathbf{v}\|}, \frac{\omega_{\text{max}}}{\|\boldsymbol{\omega}\|}\right), \quad \mathbf{v} \leftarrow \sigma \mathbf{v}, \quad \boldsymbol{\omega} \leftarrow \sigma \boldsymbol{\omega}, \quad (9)$$

with $v_{\text{max}} = \text{max_lin_vel}$ (0.15 m/s), $\omega_{\text{max}} = \text{max_rot_vel}$ (0.75 rad/s). The twist $(\mathbf{v}, \boldsymbol{\omega})$ is the command handed to the backend.

5.2 Servo mode (`--mode servo`, RTDE only)

Instead of a velocity, a clamped absolute target pose is streamed. A per-cycle displacement clamp (after the Open-Teach/deoxys operator) bounds the step:

$$\mathbf{p}_{\text{cmd}} = \mathbf{p}_c + \text{clip}(\mathbf{p}_{\text{tgt}} - \mathbf{p}_c, \delta_p), \quad (10)$$

$$\boldsymbol{\omega}_{\text{step}} = \text{clip}(\text{rotvec}(\mathbf{q}_{\text{tgt}} \otimes \mathbf{q}_c^*), \delta_r), \quad \mathbf{q}_{\text{cmd}} = \text{rotvec}^{-1}(\boldsymbol{\omega}_{\text{step}}) \otimes \mathbf{q}_c, \quad (11)$$

where $\text{clip}(\mathbf{x}, \delta)$ scales $\mathbf{x}$ to norm $\leq \delta$ (`scale_to_max_norm`). Defaults $\delta_p = v_{\text{max}}\Delta t$, $\delta_r = \omega_{\text{max}}\Delta t$ (`--max-pos-step`, `--max-rot-step`). The pose $(\mathbf{p}_{\text{cmd}}, \text{rotvec}(\mathbf{q}_{\text{cmd}}))$ is the command, with servo parameters `--lookahead-time` (0.1) and `--servo-gain` (300).

6 Backend realisation

Both backends receive the *same* command from the loop above and only differ in how they apply it.

RTDE backend (real robot / URSim).

- Speed: `speedL([v; ω], a, 2Δt)`.
- Servo: `servoL([pcmd; rotvec(qcmd)], 0, 0, Δt, tlook, gain)`.

ROS backend (launch_all.sh simulation, speed mode only). The Cartesian twist is mapped to joint velocities through the UR5 geometric Jacobian $J(\mathbf{q}) \in \mathbb{R}^{6 \times 6}$ (`kinematics.ur5_fk_and_jacobian`) using damped least squares (`cartesian_to_joint_vel`):

$$\dot{\mathbf{q}} = J^\top (JJ^\top + \lambda^2 I)^{-1} \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix}, \quad \lambda = \text{damping} (0.05), \quad (12)$$

followed by a joint-speed clamp to `max_joint_vel` (1.5 rad/s), direction preserved. $\dot{\mathbf{q}}$ is published as a `Float64MultiArray` on `/forward_velocity_controller/commands`. Servo mode is unavailable here (no `servoL` equivalent) and is rejected at the CLI.

7 Safety

- **Grip deadman:** motion only while the grip button is held; releasing it re-baselines the origins on the next press.
- **Reader watchdog:** `oculus_reader` serves cached poses with no timestamp, so staleness is detected by change-tracking the raw matrix. If the controller pose is unchanged for `> watchdog_timeout` (0.25s) while enabled, motion is stopped and a latch requires a deliberate grip re-press.
- **Stop ordering:** the mode-appropriate stop (`servoStop` in servo mode, else `speedStop`; zeros on the ROS backend) is issued on grip release, watchdog trip, and shutdown, *before* `stopScript` and peripheral teardown. The ROS `forward_velocity_controller` holds the last command with no auto-decay, so publishing zeros is mandatory to stop.

8 Symbol / flag / code reference

Symbol	Meaning	CLI flag	Default
s_{pos}	position scale	<code>--pos-scale</code>	1.0
s_{prec}	precision multiplier	<code>--precision-scale</code>	0.5
α	filter constant	<code>--filter-alpha</code>	0.8
k_p, k_r	P-gains	<code>--pos-gain, --rot-gain</code>	3.0, 2.0
$v_{\text{max}}, \omega_{\text{max}}$	velocity limits	<code>--max-lin-vel, --max-rot-vel</code>	0.15, 0.75
δ_p, δ_r	servo step clamps	<code>--max-pos-step, --max-rot-step</code>	$v_{\text{max}}\Delta t, \omega_{\text{max}}\Delta t$
t_{look}	servoL lookahead	<code>--lookahead-time</code>	0.1
gain	servoL gain	<code>--servo-gain</code>	300
λ	IK damping (ROS)	<code>--damping</code>	0.05
M	axis remap	<code>--rmat-reorder</code>	$[-2, -1, -3, 4]$